

Práctica Evaluable - Unidad 3

Generador de Titulares con Miniature GPT

Información General

Campo	Valor
Unidad	3 - Arquitectura Transformers y Acceso Programático
Tipo	Práctica individual
Duración estimada	120 minutos
Entrega	Notebook Jupyter (.ipynb) o enlace a Google Colab
Fecha límite	Según calendario del curso

Objetivo

Implementar un modelo Transformer desde cero para generar titulares de noticias en español, consolidando los conceptos teóricos de la arquitectura Transformer aplicándolos en código real.

Basado en: Tutorial de Keras "Text generation with a miniature GPT"

- Artículo: https://keras.io/examples/generative/text_generation_with_miniature_gpt/
- Código: https://github.com/keras-team/keras-io/blob/master/examples/generative/text_generation_with_miniature_gpt.py

Objetivos de Aprendizaje

1. Implementar los componentes clave de un Transformer (embeddings, atención, FFN)
2. Entrenar un modelo de lenguaje desde cero
3. Experimentar con generación de texto y parámetros
4. Comprender la arquitectura a nivel de código

Dataset

El dataset contiene **1,079 titulares en español** descargados de fuentes periodísticas. Cada línea del archivo es un titular independiente. Este corpus compacto permite entrenar un modelo de lenguaje a nivel de carácter en un tiempo razonable.

Parte 1: Preparación del Entorno (10 min)

Contexto

Antes de construir el modelo, necesitamos configurar el entorno de trabajo, importar las librerías necesarias y descargar el dataset. Utilizaremos **Google Colab con GPU** para acelerar el entrenamiento.

Instrucciones

1. Abre un nuevo notebook en Google Colab
2. Activa la GPU: Entorno de ejecución > Cambiar tipo de entorno de ejecución > GPU
3. Ejecuta el siguiente código para verificar la GPU, importar librerías y descargar el dataset

Código

```
# Verificar GPU en Colab
import tensorflow as tf
print("GPU disponible:", tf.config.list_physical_devices('GPU'))

# Imports
import numpy as np
import keras
from keras import layers

# Descargar dataset
!gdown 199dxi24ln2b-_S4mhH2sgpr3nvxmoxZN -O titulares.txt

# Cargar texto
with open('titulares.txt', 'r', encoding='utf-8') as f:
    text = f.read()

print(f"Caracteres totales: {len(text)}")
print(f"Muestra:\n{text[:500]}")
```

Nota: Si `gdown` no está instalado, ejecuta `!pip install gdown` primero.

Parte 2: Tokenización a Nivel de Carácter (10 min)

Contexto

La tokenización es el proceso de convertir texto en una secuencia de números que el modelo pueda procesar. Existen diferentes estrategias:

- **Nivel de carácter:** Cada carácter individual es un token. Vocabulario pequeño (~70-100 tokens), pero secuencias largas.
- **Nivel de subpalabra (BPE/WordPiece):** Divide las palabras en fragmentos frecuentes. Es el método usado por GPT, BERT y la mayoría de LLMs modernos.
- **Nivel de palabra:** Cada palabra completa es un token. Vocabulario muy grande, no maneja palabras desconocidas.

En esta práctica usamos **tokenización a nivel de carácter** porque simplifica la implementación y permite comprender el flujo completo sin depender de tokenizadores externos.

Código

```
# Crear vocabulario
vocab = sorted(set(text))
vocab_size = len(vocab)
print(f"Vocabulario: {vocab_size} caracteres únicos")

# Mapeos
char_to_idx = {ch: i for i, ch in enumerate(vocab)}
idx_to_char = {i: ch for i, ch in enumerate(vocab)}

def encode(s):
    return [char_to_idx[c] for c in s]

def decode(ids):
    return ''.join([idx_to_char[i] for i in ids])

# Test
print(encode("Hola"))
print(decode([27, 52, 47, 38]))
```

Reflexión: Con tokenización a nivel de carácter, el modelo debe aprender a formar palabras desde cero. Esto es más difícil que la tokenización por subpalabras, pero nos permite ver cómo el modelo aprende patrones lingüísticos desde lo más básico.

Parte 3: Preparar Datos de Entrenamiento (15 min)

Contexto

Para entrenar un modelo de lenguaje autoregresivo, la entrada es una secuencia de tokens y la salida esperada es la misma secuencia desplazada una posición a la derecha. Esto se conoce como **teacher forcing**: en cada posición, el modelo recibe el token correcto como entrada y debe predecir el siguiente.

Por ejemplo, si la secuencia es "Hola" → [H, o, l, a]:

- **Entrada (X):** [H, o, l]
- **Salida (y):** [o, l, a]

El modelo aprende a predecir el siguiente carácter dado el contexto anterior.

Código

```
# Parámetros
SEQ_LENGTH = 80
BATCH_SIZE = 64

# Tokenizar todo
tokens = np.array(encode(text))

# Crear secuencias X, y
def crear_secuencias(tokens, seq_len):
```

```
X, y = [], []
for i in range(len(tokens) - seq_len):
    X.append(tokens[i:i+seq_len])
    y.append(tokens[i+1:i+seq_len+1])
return np.array(X), np.array(y)

X, y = crear_secuencias(tokens, SEQ_LENGTH)
print(f"Secuencias: {X.shape}")

# Dataset de TensorFlow
dataset = tf.data.Dataset.from_tensor_slices((X, y))
dataset =
dataset.shuffle(10000).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

Importante: `SEQ_LENGTH = 80` define la ventana de contexto del modelo. Un titular típico tiene entre 40 y 100 caracteres, por lo que 80 es un valor razonable. `BATCH_SIZE = 64` agrupa las secuencias para entrenamiento eficiente en GPU.

Parte 4: Componentes del Transformer (25 min)

Contexto

Esta es la parte central de la práctica. Implementaremos los dos componentes fundamentales de la arquitectura Transformer tal como se estudiaron en la sesión teórica:

1. **Token & Position Embedding:** Convierte tokens e índices de posición en vectores densos.
2. **Transformer Block:** Contiene atención multi-cabeza con máscara causal, red feed-forward y conexiones residuales con normalización.

4.1 Embeddings con Posición

Los Transformers no tienen recurrencia ni convoluciones, por lo que necesitan **embeddings posicionales** para saber el orden de los tokens. Sumamos el embedding del token con el embedding de su posición.

```
class TokenAndPositionEmbedding(layers.Layer):
    def __init__(self, maxlen, vocab_size, embed_dim):
        super().__init__()
        self.token_emb = layers.Embedding(input_dim=vocab_size,
output_dim=embed_dim)
        self.pos_emb = layers.Embedding(input_dim=maxlen,
output_dim=embed_dim)

    def call(self, x):
        maxlen = tf.shape(x)[-1]
        positions = tf.range(start=0, limit=maxlen, delta=1)
        positions = self.pos_emb(positions)
        x = self.token_emb(x)
        return x + positions
```

Conexión con la teoría: En la sesión 1 vimos que los embeddings posicionales pueden ser fijos (sinusoidales, como en el paper original "Attention is All You Need") o aprendidos. Aquí usamos **embeddings posicionales aprendidos**, que es la estrategia utilizada por GPT.

4.2 Bloque Transformer con Atención Causal

El bloque Transformer implementa la secuencia completa: **Multi-Head Attention → Add & Norm → Feed-Forward → Add & Norm**. La máscara causal es esencial para modelos generativos: impide que cada posición "vea" tokens futuros.

```
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, dropout=0.1):
        super().__init__()
        self.att = layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)
        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="gelu"),
            layers.Dense(embed_dim),
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(dropout)
        self.dropout2 = layers.Dropout(dropout)

    def causal_attention_mask(self, batch_size, seq_len):
        """Máscara para que cada posición solo vea anteriores."""
        i = tf.range(seq_len)[:, tf.newaxis]
        j = tf.range(seq_len)
        mask = tf.cast(i >= j, dtype=tf.float32)
        mask = tf.reshape(mask, [1, 1, seq_len, seq_len])
        return tf.tile(mask, [batch_size, 1, 1, 1])

    def call(self, inputs, training):
        batch_size = tf.shape(inputs)[0]
        seq_len = tf.shape(inputs)[1]
```

```
mask = self.causal_attention_mask(batch_size, seq_len)

attn_output = self.att(inputs, inputs, attention_mask=mask)
attn_output = self.dropout1(attn_output, training=training)
out1 = self.layer_norm1(inputs + attn_output)

ffn_output = self.ffn(out1)
ffn_output = self.dropout2(ffn_output, training=training)
return self.layer_norm2(out1 + ffn_output)
```

Conexión con la teoría:

- **Multi-Head Attention:** Permite al modelo atender a diferentes partes de la secuencia simultáneamente. Cada "cabeza" puede capturar relaciones distintas (sintácticas, semánticas, etc.).
- **Máscara causal:** La matriz triangular inferior garantiza que la posición i solo puede atender a las posiciones $0, 1, \dots, i$. Esto es lo que hace que el modelo sea autoregresivo.
- **Feed-Forward Network (FFN):** Dos capas densas con activación GELU. Procesa cada posición de forma independiente, añadiendo capacidad de transformación no lineal.
- **Layer Normalization + Residual:** Estabilizan el entrenamiento y permiten apilar múltiples bloques sin que los gradientes desaparezcan.

Parte 5: Modelo Completo (10 min)

Contexto

Ahora ensamblamos los componentes en un modelo completo. Apilamos varios bloques Transformer y añadimos una capa de salida que predice la distribución de probabilidad sobre todo el vocabulario para cada posición.

Hiperparámetros

Parámetro	Valor	Descripción
EMBED_DIM	256	Dimensión de los embeddings
NUM_HEADS	4	Cabezas de atención (cada una de dim 64)
FF_DIM	512	Dimensión interna de la FFN
NUM_BLOCKS	4	Número de bloques Transformer apilados

Código

```
EMBED_DIM = 256
NUM_HEADS = 4
FF_DIM = 512
NUM_BLOCKS = 4
```

```
def crear_modelo():
    inputs = layers.Input(shape=(SEQ_LENGTH,), dtype=tf.int32)
    x = TokenAndPositionEmbedding(SEQ_LENGTH, vocab_size, EMBED_DIM)
    (inputs)
    for _ in range(NUM_BLOCKS):
        x = TransformerBlock(EMBED_DIM, NUM_HEADS, FF_DIM)(x)
    outputs = layers.Dense(vocab_size, activation="softmax")(x)
    return keras.Model(inputs=inputs, outputs=outputs)

model = crear_modelo()
model.summary()
```

Nota: La capa final `Dense(vocab_size, activation="softmax")` convierte la representación interna en una distribución de probabilidad sobre los caracteres del vocabulario. Para cada posición de la secuencia, el modelo predice qué carácter viene a continuación.

Parte 6: Entrenamiento (15 min)

Contexto

Entrenamos el modelo usando **sparse categorical crossentropy** como función de pérdida (ideal cuando las etiquetas son índices enteros, no one-hot). Utilizamos callbacks para detener el entrenamiento si no mejora (EarlyStopping) y para reducir la tasa de aprendizaje si la pérdida se estanca (ReduceLROnPlateau).

Código

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

callbacks = [
    keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=2)
]

history = model.fit(dataset, epochs=30, callbacks=callbacks)
```

Visualización de Curvas de Entrenamiento

Después del entrenamiento, genera las gráficas de pérdida y precisión para documentar el proceso:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
# Pérdida
axes[0].plot(history.history['loss'], label='Loss')
axes[0].set_title('Pérdida durante el entrenamiento')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True)

# Precisión
axes[1].plot(history.history['accuracy'], label='Accuracy', color='green')
axes[1].set_title('Precisión durante el entrenamiento')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

Esperable: La pérdida debería descender progresivamente y la precisión debería aumentar. Si el modelo no converge, prueba a ajustar la tasa de aprendizaje o el número de épocas.

Parte 7: Generación de Texto (20 min)

Contexto

La generación de texto se realiza de forma **autoregresiva**: el modelo predice un carácter, lo añade a la secuencia, y vuelve a predecir el siguiente. El parámetro **temperatura** controla la aleatoriedad:

- **Temperatura baja (0.5):** Más conservador, elige los caracteres más probables. Texto más coherente pero repetitivo.
- **Temperatura media (1.0):** Balance entre creatividad y coherencia.
- **Temperatura alta (1.5):** Más aleatorio, puede generar combinaciones novedosas pero también incoherentes.

Código

```
def generar_texto(model, inicio, longitud=100, temperatura=1.0):
    """Genera texto de forma autoregresiva."""
    generado = list(encode(inicio))

    for _ in range(longitud):
        input_seq = generado[-SEQ_LENGTH:]
        input_seq = np.array(input_seq)[np.newaxis, :]

        if len(input_seq[0]) < SEQ_LENGTH:
            pad_len = SEQ_LENGTH - len(input_seq[0])
            input_seq = np.pad(input_seq, ((0,0), (pad_len, 0)))
```

```

    preds = model.predict(input_seq, verbose=0)[0, -1, :]

    preds = np.log(preds + 1e-10) / temperatura
    preds = np.exp(preds) / np.sum(np.exp(preds))

    next_idx = np.random.choice(len(preds), p=preds)
    generado.append(next_idx)

    if idx_to_char[next_idx] == '\n':
        break

    return decode(generado)

# Generar con diferentes temperaturas
print("=== Temperatura 0.5 ===")
print(generar_texto(model, "El gobierno ", temperatura=0.5))

print("\n=== Temperatura 1.0 ===")
print(generar_texto(model, "El gobierno ", temperatura=1.0))

print("\n=== Temperatura 1.5 ===")
print(generar_texto(model, "El gobierno ", temperatura=1.5))

```

Experimentación Adicional

Prueba con diferentes textos de inicio para explorar lo que el modelo ha aprendido:

```

inicios = ["La economía ", "Un nuevo ", "El presidente ", "Argentina ", "Se
espera "]

for inicio in inicios:
    print(f"Inicio: '{inicio}'")
    print(f" → {generar_texto(model, inicio, temperatura=0.8)}")
    print()

```

Documenta tus observaciones: ¿Qué patrones ha capturado el modelo? ¿Genera palabras reales en español? ¿Los titulares tienen estructura coherente? ¿Cómo cambia el resultado con diferentes temperaturas?

Recomendaciones

- Utiliza **Google Colab con GPU** activada para acelerar el entrenamiento
- Asegúrate de que el notebook se ejecuta **de principio a fin** sin errores
- Incluye **celdas Markdown** explicando cada sección en tus propias palabras
- Guarda las **gráficas de entrenamiento** (loss y accuracy)
- Experimenta con al menos **3 temperaturas** distintas (0.5, 1.0, 1.5)
- Genera **múltiples ejemplos** de titulares con diferentes textos de inicio

- Documenta cualquier **modificación** que realices a los hiperparámetros
- Si el modelo no converge, prueba a ajustar: tasa de aprendizaje, número de bloques, dimensión de embeddings o número de épocas

Rúbrica de Evaluación

Criterio (Peso)	Excelente	Satisfactorio	Insuficiente
Código funcional (30% - 3 pts)	3-2.4 pts: Notebook se ejecuta completamente sin errores	2.3-1.5 pts: Funciona con errores menores	1.4-0 pts: No se ejecuta o errores críticos
Implementación del Transformer (30% - 3 pts)	3-2.4 pts: Implementación correcta de embeddings, atención causal y bloques	2.3-1.5 pts: Mayormente correcta con pequeñas imprecisiones	1.4-0 pts: Componentes mal implementados o faltantes
Experimentación documentada (20% - 2 pts)	2-1.6 pts: Gráficas de entrenamiento y múltiples ejemplos con temperaturas	1.5-1 pts: Experimentación básica con documentación parcial	0.9-0 pts: Sin experimentación
Análisis y reflexión (20% - 2 pts)	2-1.6 pts: Análisis profundo de patrones y limitaciones	1.5-1 pts: Reflexiones básicas pero correctas	0.9-0 pts: Sin análisis

Formato y Proceso de Entrega

Estructura del Notebook

1. Parte 1: Preparación del Entorno
 - Verificación de GPU
 - Imports y descarga del dataset
2. Parte 2: Tokenización a Nivel de Carácter
 - Vocabulario y mapeos
 - Funciones encode/decode
3. Parte 3: Preparar Datos de Entrenamiento
 - Creación de secuencias X, y
 - Dataset de TensorFlow
4. Parte 4: Componentes del Transformer
 - TokenAndPositionEmbedding
 - TransformerBlock con atención causal
5. Parte 5: Modelo Completo
 - Ensamblaje y resumen del modelo

- 6. Parte 6: Entrenamiento
 - Compilación y entrenamiento
 - Gráficas de loss/accuracy
- 7. Parte 7: Generación de Texto
 - Función de generación
 - Ejemplos con temperaturas 0.5, 1.0, 1.5
 - Análisis y reflexión

Requisitos Técnicos

- **Formato:** Notebook Jupyter (.ipynb) o enlace compartido a Google Colab
- **Nombre del archivo:** `Apellido_Nombre_U3_Practica.ipynb`
- **Contenido requerido:**
 1. Código completo y funcional de todas las partes
 2. Resultados de entrenamiento (gráficas de loss/accuracy)
 3. Ejemplos de titulares generados con diferentes temperaturas (0.5, 1.0, 1.5)
 4. Análisis en celdas Markdown: ¿Qué patrones capturó el modelo? ¿Cuáles son sus limitaciones?

Proceso de Entrega

1. Descargar el notebook desde Google Colab (Archivo > Descargar > Descargar .ipynb)
2. Verificar que el notebook incluye las salidas de las celdas ejecutadas
3. Subir en Blackboard antes de la fecha límite
4. Verificar que la entrega se ha realizado correctamente

Recursos Útiles

Herramientas

- [Google Colab](#)
- [Keras Documentation](#)
- [TensorFlow Documentation](#)

Referencias

- [Text generation with a miniature GPT \(Keras\)](#)
- [Código fuente del tutorial](#)
- [Attention Is All You Need \(paper original\)](#)
- [Sesión 1 - Teoría](#)
- [Sesión 2 - Teoría](#)

Notas Finales

- Esta práctica es **individual**

- Puedes consultar la documentación oficial de Keras y TensorFlow
- Se valora la originalidad en el análisis y las reflexiones personales
- Asegúrate de que las gráficas y salidas sean legibles
- En caso de dudas, consulta al profesor

Fecha de entrega: Consultar calendario del curso